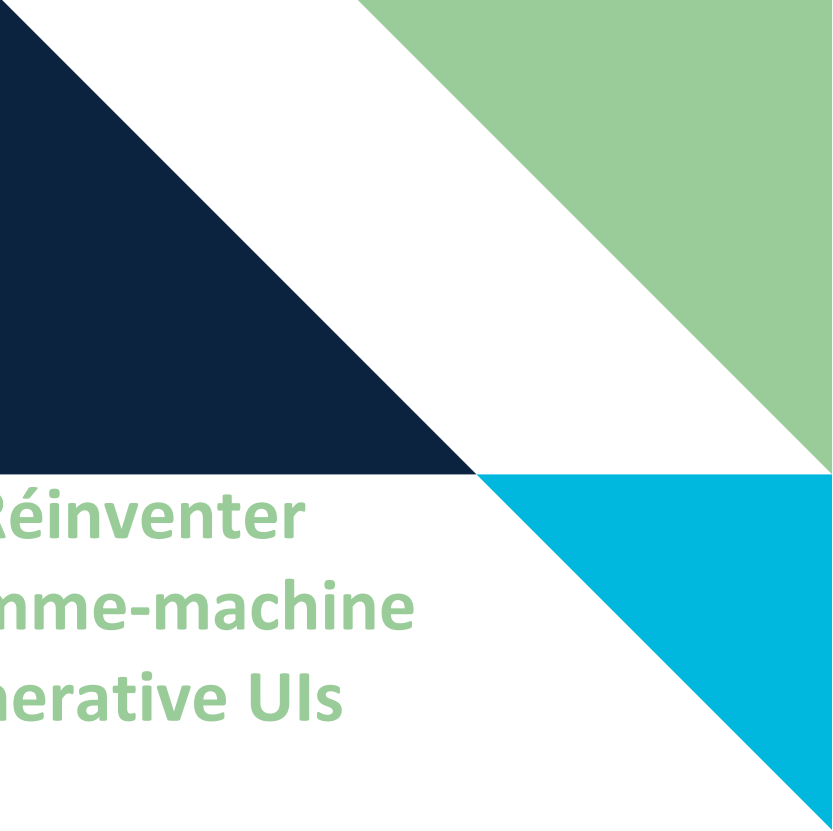


IMT Atlantique

Projet 28

Campus de Brest



UI AI Natives – Réinventer l'interaction homme-machine à travers les Generative UIs

Synthèse du lancement de projet

Version 1

Date 28/10/2026

Auteurs :

BADSSI Chaimae

BARET Alexis

BEDOUIN Arnaud

BELKHIRIA Zeineb

MOREAU Maxandre

SAFTA Yassine

Encadré par : STOENESCU Daniel

Table de matière

1. Introduction

- 1.1 Objet du document
- 1.2 Terminologie

2. Présentation du projet

- 2.1 Contexte
- 2.2 Client
- 2.3 Finalité
- 2.4 Mission
- 2.5 Objectifs
- 2.6 Enjeux
- 2.7 Évolution scientifique et technologique
- 2.8 Liste des parties prenantes
- 2.9 Problématique

3. État de l'art

- 3.1 Méthodologie
- 3.2 Protocoles Agent – Interface
- 3.3 Protocoles Agent – Outils
 - 3.3.1 MCP
 - 3.3.2 UTCP
 - 3.3.3 WebMCP
 - 3.3.4 Principes de sécurité
- 3.4 Protocoles Agent – Agent
- 3.5 Frameworks d'interface génératives
- 3.6 Frameworks d'orchestration
- 3.7 LLM
- 3.8 Outils de robustesse
 - 3.8.1 Validation des entrées
 - 3.8.2 Contrôle des outils
 - 3.8.3 Gestion des erreurs
 - 3.8.4 Confirmation utilisateur
 - 3.8.5 Traçabilité
- 3.9 Synthèse

4. Cahier des charges fonctionnel

- 4.1 Contexte d'utilisation du système
- 4.2 Expression fonctionnelle du besoin
 - 4.2.1 Fonctions de service et de contrainte
 - 4.2.2 Scénarios
- 4.3 Validation du besoin

5. Plan de travail

- 5.1 Compréhension du besoin et cadrage du projet
- 5.2 Exploration de l'état de l'art
- 5.3 Identification et sélection des cas d'usage
- 5.4 Conception fonctionnelle et architecture technique
- 5.5 Réalisation et expérimentation du prototype
- 5.6 Bilan et perspectives

6. Organisation

- 6.1 Présentation de l'équipe
- 6.2 Structure du projet et déroulement temporel
- 6.3 Décomposition du projet en lots et tâches
- 6.4 Planification et critères de fin de lot
- 6.5 Ressources et suivi de l'avancement
- 6.6 Jalons et livrables
- 6.7 Diagramme de Gantt et dépendances
- 6.8 Identification des risques
- 6.9 Méthode de cotation
- 6.10 Modalités de suivi
- 6.11 AMDEC et tableau des risques
- 6.12 AMDEC et tableau des risques (suite)

7. Validation du projet

8. Références bibliographiques

1 Introduction

1.1 Objet du document

Ce document constitue la synthèse du lancement du projet n° 28, « UI AI Natives – Réinventer l'interaction homme-machine à travers les Generative UIs », réalisé dans le cadre des projets PRONTO d'IMT Atlantique (campus de Brest). Ce projet est proposé par Sopra Steria, entreprise de conseil et de services numériques, et plus précisément par son agence de Brest (Guipavas), rattachée à la verticale Services Financiers. Il est suivi côté entreprise par M. Jérôme Gauthier, architecte IA, et encadré côté école par M. Daniel Stoenescu.

Le projet consiste à concevoir, architecturer et prototyper une application métier bancaire dont l'interface est générée dynamiquement par l'intelligence artificielle, au-delà du simple « chatbot textuel ». Ce document a pour but de formaliser la compréhension commune du besoin entre le partenaire, le tuteur et l'équipe projet, et de servir de référence pour la suite du projet. Il présente successivement le contexte et les enjeux du projet, un état de l'art des technologies d'interfaces génératives, le cahier des charges fonctionnel, puis le plan de travail, l'organisation de l'équipe et les risques identifiés, avant de préciser les modalités de validation du projet.

1.2 Terminologie

Terme	Abréviation	Description
Agent / IA agentique		Système qui s'appuie sur un LLM pour enchaîner plusieurs étapes de façon autonome afin d'atteindre un objectif.
Agent-to-Agent	A2A	Protocole de communication entre agents spécialisés.
Agent-to-User Interface	A2UI	Protocole par lequel un agent décrit, dans un format déclaratif, l'interface à afficher. L'application la rend avec ses propres composants.
Agent-User Interaction Protocol	AG-UI	Protocole qui standardise les échanges en temps réel entre un agent et l'application frontend.
AI-Native		Se dit d'une application conçue dès l'origine autour de l'IA, où le backend génère l'interface au lieu de servir des écrans figés.

Application Programming Interface	API	Outil logiciel qui permet à deux programmes informatiques de communiquer et d'échanger des données entre eux.
Backend		Partie de l'application exécutée côté serveur, qui porte la logique métier et les données.
Chatbot		Assistant conversationnel limité à un échange textuel avec l'utilisateur.
Design System		Catalogue de composants et de règles visuelle garantissant la cohérence d'une interface.
Frontend		Partie de l'application exécutée côté utilisateur (navigateur), qui affiche l'interface.
Function Calling		Capacité d'un modèle de langage (LLM) à générer des données structurées (souvent au format JSON) pour demander l'exécution d'une fonction ou d'une API externe.
Generative User Interfaces	Generative UIs	Interface numérique capable de créer ou de modifier dynamiquement ses propres composants visuels et fonctionnels en fonction des besoins de l'utilisateur.
IA générative	IA Gen	Famille de modèles d'IA capables de produire du contenu à partir d'une consigne.
JavaScript Object Notation	JSON	Format texte de données structurées.
Large Language Model	LLM	IA entraînée sur un grand nombre de données pour interpréter, générer et comprendre du texte en langage naturel.
Model Context Protocol	MCP	Protocole standard ouvert permettant à un agent de découvrir et d'appeler des outils et ressources externes.
Outil (tool)		Fonction prédéfinie exposée par l'application que l'agent peut demander à appeler.
Universal Tool Calling Protocol	UTCP	Protocole d'appel d'outils indépendant du fournisseur de LLM.
User Interface / User Experience	UI / UX	L'interface visible par l'utilisateur / la qualité globale de son expérience d'utilisation.

WebMCP		Déclinaison de MCP au Web : une page expose ses fonctionnalités comme outils utilisables par un agent.
--------	--	--

2 Présentation du projet

2.1 Contexte

L'émergence des grands modèles de langage (LLM) et de l'IA agentique transforme en profondeur la manière de concevoir les logiciels. Au-delà des assistants conversationnels, c'est désormais l'interface utilisateur elle-même qui peut être produite par l'IA : on parle d'interfaces utilisateur génératives (Generative UIs). Dans le modèle traditionnel, un frontend conçu à l'avance consomme des données statiques issues d'un backend rigide. L'approche dite AI-Native renverse cette logique : un backend propulsé par des LLM et des systèmes d'agents devient capable de générer l'interface en temps réel, en l'adaptant au contexte, au parcours et à l'intention de chaque utilisateur.

Ce changement de paradigme concerne directement le secteur de la banque et de l'assurance, où les attentes des clients en matière de simplicité et de personnalisation se heurtent à de fortes exigences de sécurité, de fiabilité et de conformité réglementaire. C'est dans ce contexte que Sopra Steria, présentée dans la section suivante, propose ce projet à travers sa verticale Services Financiers.

Le projet 28 d'IMT Atlantique consiste ainsi à concevoir, architecturer et prototyper une application métier bancaire qui dépasse le simple « chatbot textuel » pour exploiter pleinement le potentiel des interfaces génératives. Les cas d'usage (consultation de comptes, échanges avec un conseiller, souscription de services, paiements. . .) sont à définir par l'équipe et à proposer au partenaire. Le projet s'articule autour de quatre piliers : les nouveaux patterns UX, l'équilibre entre génération dynamique et cadrage de l'interface, les protocoles d'échange émergents entre LLM, outils et rendu (WebMCP, A2UI, AG-UI. . .), et enfin l'industrialisation et la robustesse de ces solutions dans un contexte bancaire.

Placée dans une situation proche de celle d'une équipe innovation, l'équipe doit partir d'un besoin métier réel, explorer des technologies émergentes, faire des choix d'architecture et aboutir à un démonstrateur permettant de rendre l'idée visible, testable et convaincante auprès de professionnels.

2.2 Client

Sopra Steria est un acteur majeur de la Tech en Europe, reconnu pour ses activités de conseil, de services et de solutions numériques. Le groupe compte environ 51 000 collaborateurs répartis dans près de 30 pays et accompagne de grandes organisations dans leur transformation numérique, en combinant innovation, expertise métier et excellence technique. Il intervient dans de nombreux secteurs : services financiers, espace, défense et sécurité, aéronautique, transport, énergie ou encore secteur public.

Le projet est porté par la verticale Services Financiers du groupe, qui travaille pour les principales banques et compagnies d'assurance françaises et européennes. Ses équipes interviennent sur des domaines métier clés tels que l'assurance, les crédits, les moyens de paiement, la conformité réglementaire, la gestion des risques et la lutte contre le blanchiment. Elles y mènent des projets de modernisation des systèmes d'information, de cloud, de data, de cybersécurité, d'automatisation et d'intégration de l'intelligence artificielle, aussi bien dans les méthodes de développement que dans les parcours utilisateurs.

Implantée depuis plusieurs années à Brest et dans l'Ouest (Nantes, Rennes), Sopra Steria y conduit des projets stratégiques pour les acteurs bancaires locaux. C'est l'agence de Brest, située à Guipavas, qui est à l'origine de ce projet. Nos interlocuteurs y sont M. Florian Boucher, directeur des opérations du domaine Services Financiers, correspondant administratif du projet, et M. Jérôme Gauthier, architecte IA, qui assure le suivi de l'équipe.

2.3 Finalité

Le projet vise à explorer et démontrer le potentiel des interfaces utilisateur génératives dans le secteur bancaire, afin de préparer le passage d'applications aux parcours figés vers des applications AI-Native capables d'adapter dynamiquement leur interface à chaque utilisateur. À plus long terme, il doit permettre à Sopra Steria et à ses clients bancaires d'évaluer la faisabilité, l'intérêt et les limites de cette approche, pour concevoir des services financiers plus simples, plus personnalisés et plus accessibles, sans compromettre la sécurité ni la conformité.

2.4 Mission

L'équipe a pour mission de concevoir, architecturer et prototyper une application métier bancaire fondée sur les interfaces génératives, qui dépasse le simple « chatbot textuel ». Concrètement, elle doit identifier et proposer des cas d'usage pertinents au partenaire, étudier les technologies et protocoles émergents du domaine, faire et justifier des choix d'architecture, puis réaliser un démonstrateur fonctionnel couvrant les quatre piliers présentés en section 2.1.

2.5 Objectifs

Objectifs
Rendre les livrables
Mener à bien le projet
Respecter un Design System

2.6 Enjeux

Le projet ouvre la voie à une nouvelle génération d'applications bancaires, dans lesquelles l'interface n'est plus conçue une fois pour toutes mais générée à la volée selon le besoin du client.

Pour Sopra Steria, un démonstrateur convaincant constitue un support concret pour proposer de nouvelles offres à ses clients bancaires et assurantiels : parcours personnalisés, assistance hybride entre conversation et composants visuels, réduction du nombre d'écrans à concevoir et à maintenir. Pour le client final, l'enjeu est d'accéder à des services plus simples, plus rapides et plus adaptés à sa situation.

2.7 Évolution scientifique et technologique

Les interfaces génératives constituent un domaine émergent, dont les standards et bonnes pratiques sont encore en cours de définition. Le projet permet d'expérimenter des protocoles récents (WebMCP, A2UI, AG-UI. . .) et d'évaluer concrètement leur maturité. Il contribue également à des questions encore ouvertes : la juste articulation entre conversation et composants visuels, la frontière entre autonomie du modèle et cadrage de l'interface, ou encore la validation automatique d'interfaces générées. Les enseignements tirés pourront alimenter la réflexion et l'expertise de Sopra Steria sur ces sujets.

2.8 Liste des parties prenantes

Partie prenante	Rôle	Contact (le cas échéant)
Client (Sopra Steria)	Porte le besoin, valide les livrables	jerome.gauthier@soprasteria.com
Tuteur	Encadre la démarche et évalue	daniel.stoenescu@imt-atlantique.fr
Équipe projet	Mène le projet et réalise le prototype	
Utilisateurs	Clients de la banque	
Conseillers	Conseillers bancaires	
Fournisseurs de LLM	Fournit des LLM	

2.9 Problématique

Comment concevoir une application bancaire dont l'interface est générée dynamiquement par l'IA, tout en garantissant une expérience utilisateur cohérente, sûre et conforme aux exigences du secteur financier ?

3 État de l'art

3.1 Méthodologie

Cet état de l'art a pour but de présenter les grandes familles de technologies sur lesquelles peut s'appuyer une application bancaire fondée sur les interfaces génératives. À ce stade du projet, il s'agit d'un panorama : nous cherchons à comprendre le rôle de chaque brique et la manière dont elles s'articulent, et non à les comparer en détail.

Les technologies sont présentées par couche d'architecture : les protocoles reliant l'agent à l'interface, aux outils et aux autres agents, les frameworks d'interfaces génératives et d'orchestration, les LLM, puis les mécanismes de robustesse. Pour chaque couche, nous décrivons son principe et son intérêt pour notre démonstrateur bancaire.

Ce panorama s'appuie sur la documentation des technologies étudiées et sur les sources transmises par Sopra Steria. Il sera complété, dans la suite du projet, par une comparaison des solutions selon leur maturité, leur sécurité, leur facilité d'intégration et leurs performances, appuyée sur des essais pratiques, afin de justifier les choix techniques du démonstrateur.

3.2 Protocoles Agent – Interface

Dans une application classique, l'interface est conçue à l'avance : les écrans, boutons, formulaires et tableaux sont définis par les développeurs.

Avec une **interface générative**, le principe est différent. L'utilisateur formule une demande en langage naturel et l'application peut adapter dynamiquement l'interface affichée en fonction du contexte.

Par exemple, dans notre démonstrateur bancaire, un utilisateur pourrait demander :

« Je veux comprendre mes dépenses du mois dernier. »

Au lieu de simplement retourner une réponse textuelle, l'application peut construire une interface adaptée : résumé des dépenses, graphique, catégories principales ou possibilité d'approfondir certaines opérations.

L'enjeu est donc de permettre au modèle d'IA de **décrire ce qui doit être affiché**, sans pour autant lui donner directement le contrôle de toute l'application.

Deux protocoles récents répondent à ce besoin. A2UI (Agent-to-User Interface), proposé par Google, permet à l'agent de décrire l'interface à afficher dans un format déclaratif : il ne produit pas de code, mais une liste de composants choisis dans un catalogue, que l'application affiche avec ses propres composants. AG-UI (Agent-User Interaction Protocol), issu du projet CopilotKit, standardise quant à lui les échanges en temps réel entre l'agent et le frontend : messages, appels d'outils et mises à jour d'état circulent sous forme d'événements.

Ces deux protocoles sont complémentaires : A2UI décrit ce qui doit être affiché, AG-UI transporte les échanges entre l'agent et l'application.

3.3 Protocoles Agent – Outils

Dans notre système, les agents IA disposent d'**outils** : des fonctions prédéfinies (consulter un solde, préparer un virement...) qu'ils peuvent utiliser selon la demande de l'utilisateur. L'IA ne peut pas inventer de nouvel outil, et elle ne les exécute pas elle-même : elle **demande** leur appel, et c'est l'application qui l'exécute après contrôle. Ce fonctionnement apporte une garantie de sécurité que l'IA, de nature probabiliste, ne peut pas offrir seule.

Ce mécanisme, appelé **Function Calling**, est proposé nativement par chaque fournisseur de LLM, mais sous un format qui lui est propre. Les outils développés pour un modèle ne sont donc pas directement réutilisables avec un autre. Pour pallier ce problème, des **protocoles standardisés** ont émergé afin d'uniformiser la manière dont les agents découvrent et appellent les outils. Cette section les présente.

3.3.1 MCP

MCP (Model Context Protocol) fournit un protocole standard permettant à un modèle ou à un agent de découvrir et d'utiliser des outils et des ressources externes.

Dans notre cas, on peut imaginer exposer des fonctionnalités bancaires sous forme d'outils :

- consulter un compte ;
- rechercher une transaction ;
- récupérer des informations sur un bénéficiaire ;
- préparer une opération ;
- consulter certaines données financières.

L'intérêt principal est de **standardiser l'accès aux outils**.

L'agent n'a donc pas besoin de connaître l'implémentation interne de chaque fonction. Il connaît son nom, ses paramètres et ce qu'elle permet de faire.

Un point essentiel pour notre projet est que MCP ne signifie pas que l'IA dispose automatiquement d'un accès illimité au système bancaire. L'application reste responsable des autorisations et de l'exécution réelle des opérations.

3.3.2 UTCP

UTCP (Universal Tool Calling Protocol) poursuit une idée proche : faciliter l'utilisation d'outils par différents agents et modèles sans dépendre d'un format spécifique à un fournisseur de LLM.

Son intérêt principal est la **portabilité des outils**.

Dans une architecture amenée à évoluer, le fournisseur du modèle peut changer. Il est donc intéressant de séparer :

le modèle de langage → la logique agentique → les outils métier.

Cette séparation réduit la dépendance à un fournisseur particulier.

3.3.3 WebMCP

WebMCP applique cette logique au contexte du Web.

L'objectif est de permettre à une application Web d'exposer certaines de ses fonctionnalités de manière structurée afin qu'un agent puisse les découvrir et les utiliser.

Dans notre démonstrateur, cette approche est particulièrement intéressante car l'application bancaire est elle-même une application Web.

Au lieu de demander à l'agent de connaître toute l'implémentation du frontend, certaines fonctionnalités peuvent être exposées comme des capacités utilisables par l'agent.

Par exemple :

Utilisateur → Agent → outil bancaire → application → résultat

L'agent peut ainsi demander :

« Recherche les transactions supérieures à 500 € sur les trois derniers mois. »

L'application reçoit alors une demande structurée, vérifie les paramètres et exécute l'opération.

3.3.4 Principe de sécurité

Un point important de notre architecture est la distinction entre **demander une action** et **effectuer une action**.

Le LLM peut produire :

```
tool          =          "prepare_transfer"
amount        =          500
beneficiary = "Alice"
```

Mais il ne réalise pas lui-même le virement.

L'application peut alors vérifier :

1. que l'outil existe ;
2. que les paramètres sont valides ;
3. que l'utilisateur possède les droits nécessaires ;
4. que l'opération est autorisée ;
5. qu'une confirmation utilisateur est éventuellement nécessaire.

Cette séparation est particulièrement importante pour les opérations sensibles.

3.4 Protocoles Agent – Agent

Dans une architecture plus complexe, plusieurs agents peuvent être spécialisés dans des tâches différentes.

Par exemple :

- un agent chargé de comprendre la demande de l'utilisateur ;
- un agent spécialisé dans les opérations bancaires ;
- un agent spécialisé dans l'analyse financière ;
- un agent chargé de contrôler certaines opérations.

La communication **Agent-to-Agent (A2A)** permet alors d'organiser les échanges entre ces différents agents.

L'intérêt est de ne pas créer un agent unique qui devrait tout connaître et tout gérer.

On peut par exemple avoir :

Agent principal → Agent bancaire → outil bancaire

ou :

Agent principal → Agent analyse → données → résultat

Cette architecture permet une séparation plus claire des responsabilités.

Dans un contexte bancaire, cette séparation peut également faciliter le contrôle et la traçabilité des actions.

3.5 Frameworks d'interface génératives

Les frameworks d'interface générative ont pour objectif de faciliter la création d'interfaces capables de s'adapter dynamiquement aux réponses et aux actions d'un agent.

Dans une interface traditionnelle, le développeur définit à l'avance :

- les pages ;
- les boutons ;
- les formulaires ;

- les tableaux ;
- les interactions possibles.

Dans une interface générative, certains de ces éléments peuvent être déterminés dynamiquement.

Par exemple, pour une demande d'analyse financière, l'agent pourrait déterminer qu'il est pertinent d'afficher :

- un résumé ;
- un graphique ;
- un tableau ;
- des filtres ;
- des actions complémentaires.

L'intérêt pour notre projet est donc de passer d'une logique :

Utilisateur → écran prédéfini → action

à une logique davantage orientée :

Utilisateur → agent → interface adaptée au contexte.

Cependant, la génération libre d'interfaces pose des problèmes de sécurité, de cohérence et de maintenabilité.

Nous envisageons donc le principe suivant :

L'IA choisit et compose parmi des composants contrôlés par l'application.

Elle peut ainsi apporter de la flexibilité sans avoir la possibilité de générer n'importe quel comportement.

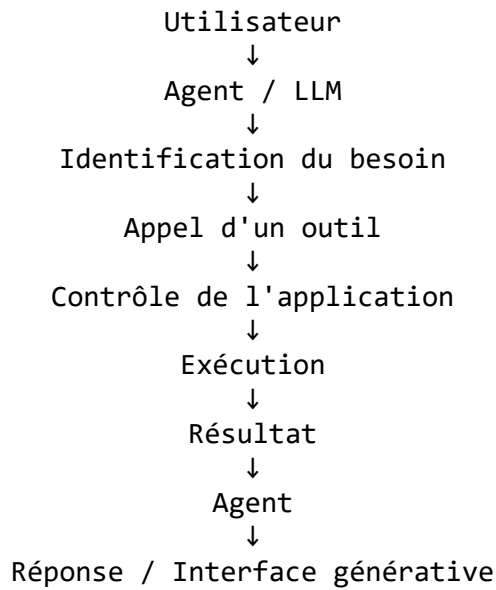
3.6 Frameworks d'orchestration

L'orchestration correspond à la manière dont les différentes briques du système sont coordonnées.

Dans notre architecture, l'orchestrateur doit notamment gérer :

1. la réception de la demande utilisateur ;
2. l'appel au LLM ;
3. l'identification des outils nécessaires ;
4. l'exécution contrôlée des outils ;
5. la récupération des résultats ;
6. le retour vers le LLM si nécessaire ;
7. la génération de la réponse ;
8. la mise à jour éventuelle de l'interface.

Un exemple simple serait :



L'orchestrateur constitue donc une couche importante entre le modèle probabiliste et les systèmes métier.

Cette séparation permet notamment de ne pas placer directement la logique métier critique dans le LLM.

3.7 LLM

Le **Large Language Model** constitue le moteur de compréhension et de génération du système.

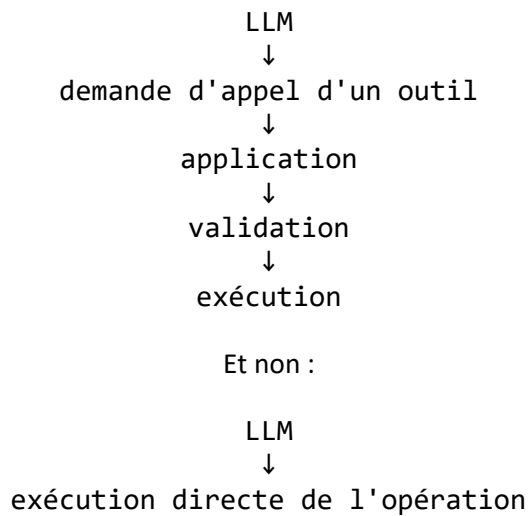
Dans notre démonstrateur, le LLM peut notamment :

- comprendre une demande formulée en langage naturel ;
- identifier l'intention de l'utilisateur ;
- déterminer les informations nécessaires ;
- sélectionner un outil ;
- générer les paramètres de cet outil ;
- interpréter le résultat obtenu ;
- générer une réponse compréhensible ;
- contribuer à déterminer l'interface à afficher.

Cependant, le LLM ne doit pas être considéré comme une source de vérité.

Il peut produire une réponse incorrecte ou sélectionner un outil de manière inadaptée. C'est pourquoi notre architecture cherche à **réduire le nombre de décisions critiques laissées au modèle**.

Par exemple, pour une opération bancaire :



Cette distinction est fondamentale pour un système bancaire.

3.8 Outils de robustesse

L'utilisation d'agents et de LLM introduit de nouvelles sources d'erreur.

Nous avons donc identifié plusieurs mécanismes permettant de rendre le système plus robuste.

3.8.1 Validation des entrées

Les paramètres générés par le LLM doivent être vérifiés avant l'exécution.

Par exemple :

amount = 500

doit être vérifié comme étant :

- un nombre ;
- positif ;
- dans une plage autorisée ;
- associé à un utilisateur autorisé.

3.8.2 Contrôle des outils

Le modèle ne doit pouvoir utiliser que les outils explicitement exposés par l'application.

3.8.3 Gestion des erreurs

Si un outil échoue, l'erreur doit être traitée par l'application plutôt que de laisser le LLM inventer un résultat.

3.8.4 Confirmation utilisateur

Pour les opérations sensibles, une confirmation explicite peut être demandée avant l'exécution.

Par exemple :

« Vous êtes sur le point de préparer un virement de 1 000 € vers Alice. Confirmez-vous cette opération ? »

3.8.5 Traçabilité

Dans un environnement bancaire, il est également important de pouvoir retrouver :

- la demande initiale ;
- l'outil sélectionné ;
- les paramètres utilisés ;
- le résultat obtenu ;
- l'action effectivement exécutée.

Cette traçabilité permet notamment de faciliter le contrôle et l'analyse des comportements du système.

3.9 Synthèse

L'étude de l'écosystème des interfaces génératives et de l'IA agentique nous a permis d'identifier une architecture reposant sur plusieurs couches complémentaires.

L'idée centrale est de **séparer l'intelligence probabiliste du LLM des fonctions critiques de l'application.**

Le LLM apporte la compréhension du langage, la capacité de raisonnement et l'adaptation de l'expérience utilisateur. L'application conserve quant à elle le contrôle des données, des outils, des autorisations et des opérations sensibles.

Cette architecture nous permet donc d'explorer les possibilités offertes par les interfaces génératives tout en tenant compte des contraintes particulières d'un environnement bancaire : **sécurité, contrôle, intégration, traçabilité et robustesse.**

Le démonstrateur réalisé dans le cadre du projet Sopra Steria vise ainsi à montrer concrètement comment une application bancaire pourrait évoluer d'une interface traditionnelle vers une expérience davantage pilotée par l'IA, tout en conservant une architecture contrôlée.

4 Cahier des charges fonctionnel

4.1 Contexte d'utilisation du système

Le produit visé est un démonstrateur d'application bancaire web (nom de travail : « Gen-UI Bank ») dont l'interface est générée en temps réel par des LLM et des agents, selon l'intention de l'utilisateur. Il s'adresse aux clients de la banque et, dans une moindre mesure, aux conseillers. Aujourd'hui, les applications bancaires ont des parcours figés et, quand il y a un assistant, c'est souvent un simple chatbot séparé de l'interface. Notre démonstrateur ne remplace rien d'existant : il sert à montrer à Sopra Steria ce que peut donner une application AI-Native.

La Figure 1 montre le système au centre et les éléments avec lesquels il échange (I1 à I8). Il faut y retenir que le LLM ne parle jamais directement à l'utilisateur ni au backend bancaire : tout passe par le système, qui contrôle les sorties avant de les afficher. Le Design System et les protocoles (A2UI, AG-UI, WebMCP...) encadrent la génération de l'interface.

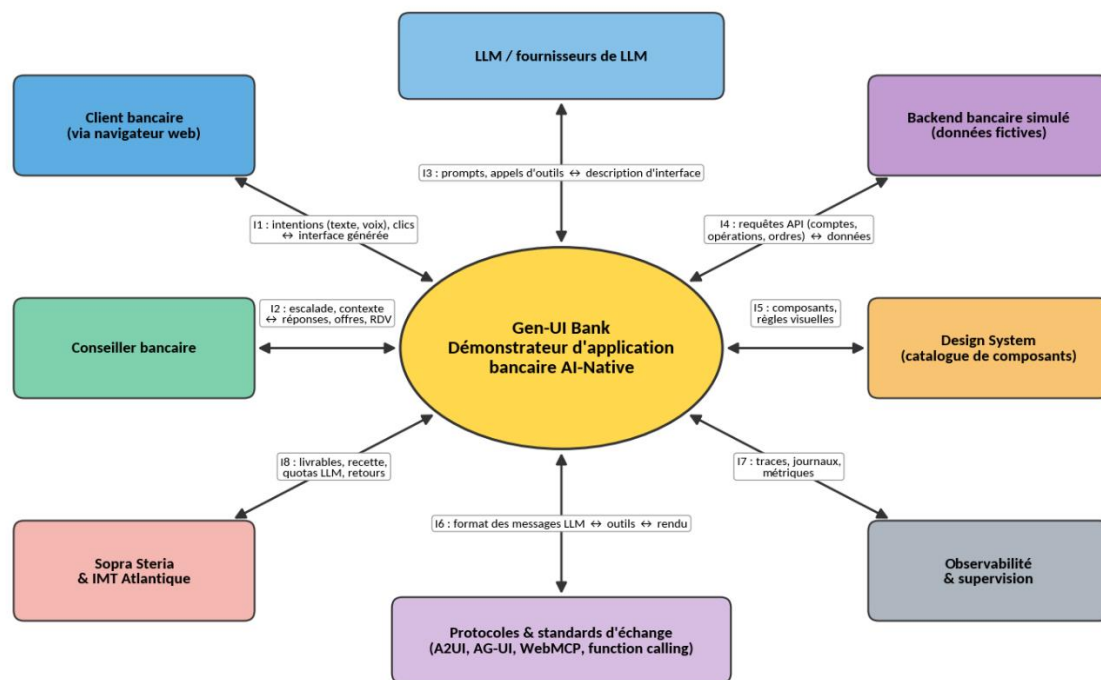


Figure 1 : Le système dans son environnement (diagramme de contexte). Les flèches à double sens sont des échanges d'information ; I1 à I8 désignent les interfaces avec chaque élément externe.

Tableau 1. Interfaces entre le système et son environnement.

Id.	Constituant externe	Nature	Flux échangés
I1	Client bancaire (navigateur web, ordinateur ou mobile)	Humaine	Vers le système : intentions (texte, voix), clics, saisies. Depuis le système : interface générée (texte, composants, graphiques).
I2	Conseiller bancaire	Humaine	Vers le système : réponses, offres, créneaux de rendez-vous. Depuis le système : demande d'escalade et résumé du contexte client.

I3	LLM / fournisseurs de LLM (accès et quotas fournis par Sopra Steria)	Logicielle (API)	Vers le LLM : prompts, contexte, outils disponibles. Depuis le LLM : texte, appels d'outils (function calling), description de l'interface à afficher.
I4	Backend bancaire simulé	Logicielle (API)	Vers le backend : requêtes (comptes, opérations, bénéficiaires, produits, ordres de virement). Depuis le backend : données fictives et résultats.
I5	Design System	Logicielle (catalogue)	Catalogue de composants et règles visuelles (couleurs, typographie, accessibilité) que toute interface générée doit respecter.
I6	Protocoles et standards d'échange (WebMCP, A2UI, AG-UI, function calling)	Normative	Format des messages entre le LLM, les outils et le rendu de l'interface.
I7	Outils d'observabilité	Logicielle	Traces, journaux et métriques émis par le système (requêtes, appels d'outils, latences, erreurs).
I8	Sopra Steria et IMT Atlantique	Organisationnelle	Livrables, démonstrations, retours et validations ; mise à disposition des accès et quotas LLM.

4.2 Expression fonctionnelle du besoin

4.2.1 Fonctions de service et de contrainte

La Figure 2 (diagramme pieuvre) résume les fonctions. Les traits bleus sont les fonctions de service (FP), qui relient deux éléments à travers le système ; les traits orange sont les contraintes (FC), qui viennent d'un seul élément extérieur. Le client est le plus sollicité (FP1 à FP5), et les contraintes importantes viennent du Design System, de la sécurité et des protocoles imposés.

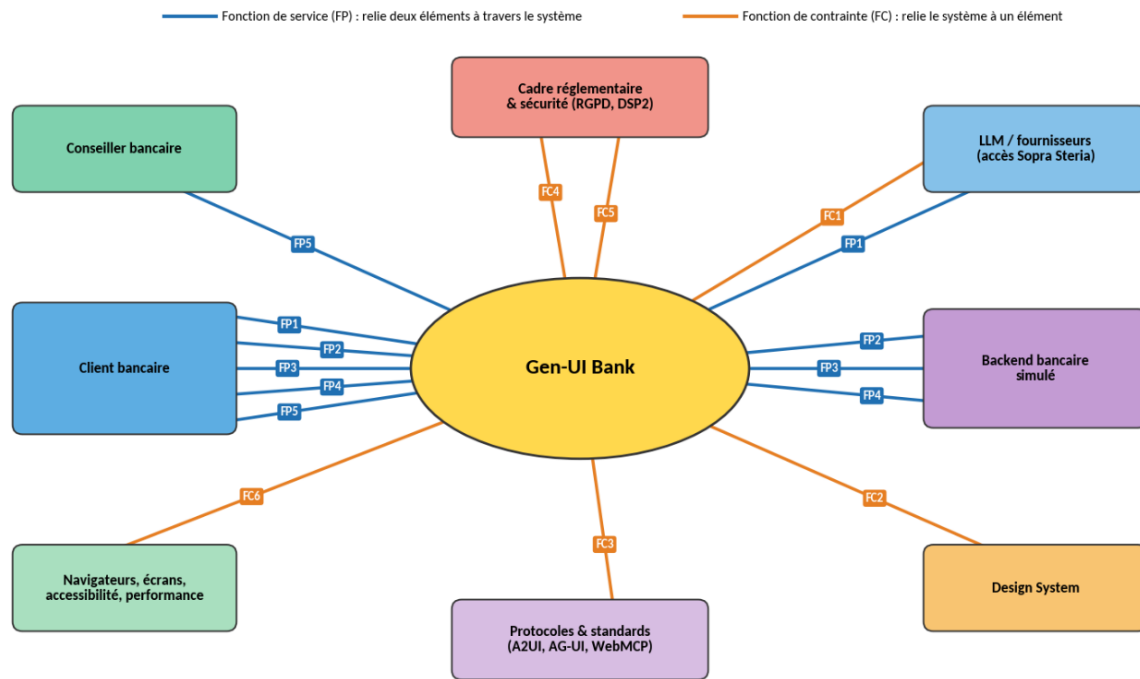


Figure 2 : Fonctions de service (FP) et de contrainte (FC) du système avec son environnement (diagramme pieuvre).

La Figure 3 présente les fonctions principales sous forme de cas d'utilisation. Le client déclenche la plupart des cas (consulter, virement, souscription, conseiller), et le conseiller intervient dans l'échange avec le client. La génération de l'interface est incluse dans les parcours de consultation, de virement et de souscription.

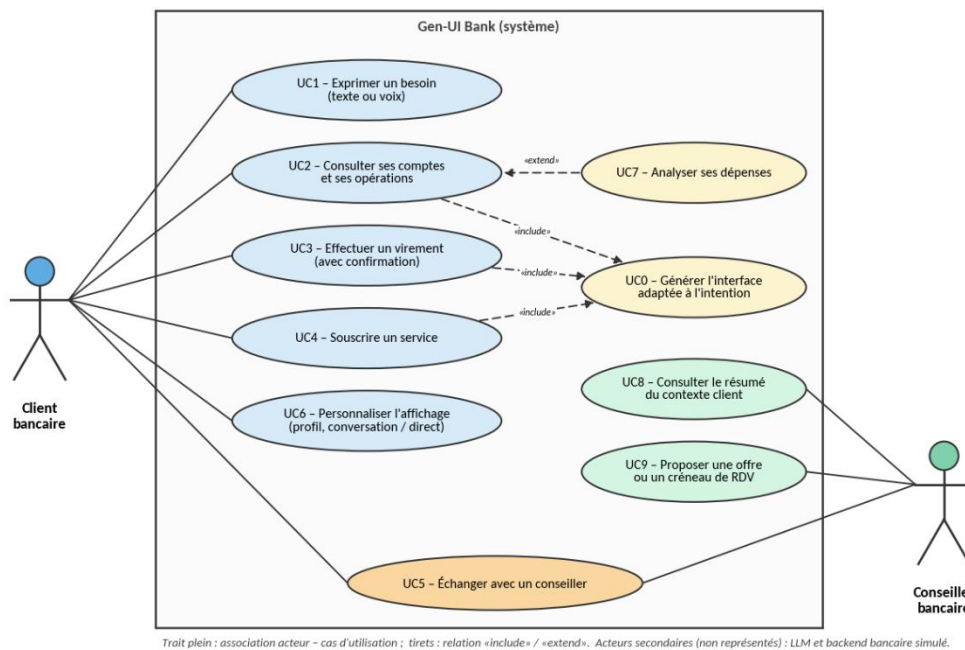


Figure 3 : Diagramme de cas d'utilisation (acteurs : client et conseiller).

Use case détaillé avec toutes les fonctionnalités : [lien usecase](#)

Le Tableau 1 liste les fonctions avec leur critère chiffré, leur flexibilité et leur priorité (1 : essentielle, 2 : importante, 3 : secondaire). (Les valeurs sont des propositions de l'équipe, à valider avec Sopra Steria.)

Tableau 1. Fonctions de service (FP, FS) et de contrainte (FC).

Fonction	Critère	Flexibilité	Priorité
FP1 – Générer une interface adaptée à l'intention du client	Premier élément affiché en	Moyenne	1
FP2 – Consulter ses comptes et ses opérations	Exactitude des montants (3 comptes, 12 mois)	Nulle	1
FP3 – Réaliser un virement	Opérations confirmées par le client avant exécution	Nulle	1
FP4 – Souscrire à un service	Champs par écran	Forte	2
FP5 – Échanger avec un conseiller	Actions pour joindre un conseiller	Moyenne	2
FS1 – Basculer entre conversation et clics	Actions faisables des deux façons	Forte	2
FS2 – Adapter l'interface au profil (ex. senior)	Profils de démonstration	Forte	3
FC1 – Utiliser les LLM fournis par Sopra Steria	LLM interchangeables	Moyenne	1
FC2 – Respecter le Design System	Composants issus du catalogue	Forte	1
FC3 – Intégrer les protocoles émergents	Protocoles implémentés (A2UI, AG-UI, WebMCP)	Moyenne	1
FC4 – Valider les sorties avant affichage	Sorties validées	Nulle	1
FC5 – Respecter confidentialité et principes réglementaires	Données réelles utilisées	Nulle	1
FC6 – Fonctionner sur les navigateurs courants et être accessible	Navigateurs ; accessibilité WCAG	Moyenne	2

Les priorités 1 vont aux fonctions qui montrent l'intérêt des interfaces génératives (FP1, FP2, FP3) et aux contraintes qui répondent à la problématique : une interface cohérente, sûre et conforme (FC2, FC4, FC5). Le virement est retenu car c'est le cas le plus sensible. La souscription et le conseiller passent en priorité 2 car ils peuvent être simplifiés si on manque de temps.

4.2.2 Scénarios

Nous avons identifié six cas d'usage : consultation de comptes, analyse des dépenses, virement, souscription d'un service, échange avec un conseiller et simulation de crédit. Les cinq premiers sont retenus pour le démonstrateur ; la simulation de crédit est une extension possible. Les scénarios suivants les racontent.

SN1 – Consulter et analyser ses dépenses. Marie demande « Combien j'ai dépensé en restaurants ces trois derniers mois ? ». Le système interroge le backend simulé et affiche un total, un graphique par mois et la liste des plus grosses dépenses. Elle clique sur un mois : la liste se filtre sans nouvel appel au LLM. Elle demande ensuite une comparaison avec l'an dernier et le graphique se complète. Avec le profil « senior », la même demande donne une interface plus simple et plus lisible. (FP1, FP2, FS1, FS2, FC2)

SN2 – Faire un virement. Marie écrit « Envoie 120 € à Paul pour le loyer ». Le système retrouve Paul, génère un formulaire pré-rempli, puis affiche un écran de confirmation qui n'est pas généré librement par le LLM. Marie confirme (avec un code fictif au-dessus de 100 €) et l'ordre est envoyé au backend par le système, jamais par le LLM. Le nouveau solde s'affiche ensuite. (FP1, FP3, FC4, FC5)

SN3 – Souscrire un service, puis parler à un conseiller. Sur son smartphone, Marie demande comment être couverte pour un voyage au Japon. Le système propose deux offres sous forme de cartes ; le formulaire ne redemande que les informations manquantes et se termine par un récapitulatif et un consentement. Si elle préfère parler à quelqu'un, elle contacte un conseiller en deux actions au plus : un résumé de la conversation lui est transmis et il peut répondre avec des créneaux de rendez-vous. (FP4, FP5, FC6)

Le Tableau 2 décrit les cas limites, c'est-à-dire ce qui doit se passer quand quelque chose ne se déroule pas comme prévu.

Tableau 2. Scénarios limites.

Situation	Comportement attendu	Fonctions
Le LLM produit une interface invalide ou inventée	La sortie est rejetée avant affichage ; une interface de repli s'affiche (en moins de 5 s)	FC2, FC4
Demande ambiguë ou hors sujet	Le système demande une précision ou refuse poliment, sans lancer d'opération	FP1
Tentative d'injection de prompt	Aucune opération n'est exécutée sans confirmation de l'utilisateur	FP3, FC4, FC5
LLM indisponible ou quota dépassé	Bascule vers un autre LLM ou mode dégradé sans génération d'interface	FC1
Valeurs hors spécifications (montant supérieur au solde, IBAN invalide)	Refus avec un message explicatif, contrôle indépendant du LLM	FP3

Chaque fonction du Tableau 1 est utilisée dans au moins un scénario.

4.3 Validation du besoin

Nous validerons le produit par une démonstration en direct devant Sopra Steria, qui reprend les scénarios.

Démonstration :

- Poser la question sur les dépenses (SN1) et vérifier le temps d'affichage et l'exactitude du total (FP1, FP2).
- Passer au profil senior et comparer l'interface (FS2, FC6).
- Réaliser le virement (SN2) puis tenter un virement supérieur au solde (FP3).
- Souscrire un service et contacter un conseiller (SN3) (FP4, FP5).
- Provoquer une sortie invalide, une injection de prompt et une coupure du LLM (FC1, FC4).

5 Le plan de travail

Afin d'atteindre les objectifs du projet, notre travail sera organisé en plusieurs étapes successives, allant de la compréhension du sujet jusqu'à la réalisation et l'évaluation d'un prototype d'application bancaire intégrant une interface AI Native.

5.1. Compréhension du sujet et définition du périmètre

Dans un premier temps, nous approfondirons la problématique des **Generative UIs** et des interfaces AI Native afin de préciser le périmètre du projet. Cette étape permettra de :

- comprendre les enjeux liés à l'évolution de l'interaction homme-machine avec l'IA générative
- identifier les besoins auxquels une interface AI Native peut répondre
- préciser les fonctionnalités attendues pour le prototype
- prendre en compte les contraintes et attentes communiquées par Sopra Steria

5.2. Exploration de l'état de l'art

Nous réaliserons ensuite une étude des solutions et approches existantes dans le domaine des interfaces AI Native et des Generative UIs.

Cette étude portera notamment sur les méthodologies, frameworks et technologies pertinents, les approches d'interface adaptative, les solutions permettant à une IA d'interagir avec des outils et services, ainsi que les architectures d'agents et de systèmes utilisant des LLM. Les ressources transmises par le client et d'autres références pertinentes alimenteront cette étude.

5.3. Identification et sélection des cas d'usage

À partir de la compréhension du besoin et de l'état de l'art, nous identifierons plusieurs **cas d'usage concrets** pour l'application bancaire.

Pour chaque cas d'usage, nous définirons plusieurs scénarios décrivant les actions de l'utilisateur et les réponses attendues, sans commencer par les choix techniques. Les cas d'usage seront ensuite analysés et priorisés afin de sélectionner ceux qui seront expérimentés dans le prototype.

5.4. Conception fonctionnelle et architecture technique

Une fois les cas d'usage sélectionnés, nous définirons les scénarios fonctionnels ainsi que l'architecture générale du prototype. Cette étape permettra notamment de :

- définir les principaux composants du système et leurs interactions
- préciser le rôle du frontend, du LLM, des agents, des outils et des API simulées
- définir les principes d'interaction entre l'utilisateur et le système
- identifier les mécanismes permettant d'adapter l'interface au contexte et au profil utilisateur
- préparer les choix techniques qui seront expérimentés lors du développement

Une attention particulière sera portée à la circulation des informations entre les composants et à la gestion du contexte transmis au LLM. Plusieurs profils utilisateurs fictifs seront également définis afin d'expérimenter l'adaptation de l'interface.

5.5. Réalisation et expérimentation du prototype

À partir de l'architecture définie, nous développerons progressivement un **prototype fonctionnel d'application bancaire intégrant une interface AI Native**, en commençant par une version minimale puis en intégrant les fonctionnalités liées à l'IA et à l'adaptation de l'interface.

Mise en place de l'application :

Nous développerons la structure générale de l'application, comprenant le frontend, les principaux composants de l'interface, les données fictives relatives aux utilisateurs et aux opérations, les API simulées ainsi que la communication entre les différents composants.

Intégration du LLM et des outils :

Nous intégrerons un modèle de langage permettant à l'utilisateur d'interagir avec l'application en langage naturel. Des outils seront mis en place afin de permettre au LLM d'accéder aux fonctionnalités de l'application, par exemple la consultation d'un compte, la recherche d'une transaction ou la préparation d'un virement.

Les mécanismes de **tool calling / function calling** seront expérimentés afin de permettre au LLM de sélectionner les outils nécessaires selon la demande de l'utilisateur.

Architecture agentique et interfaces dynamiques :

Nous expérimenterons une couche d'orchestration permettant à un agent de coordonner le LLM, les outils et les services de l'application pour les tâches nécessitant plusieurs étapes.

Nous intégrerons également les approches étudiées lors de l'état de l'art, notamment **A2UI et AG-UI**, afin d'expérimenter la génération ou l'adaptation dynamique des composants d'interface. **WebMCP** pourra également être testé comme mécanisme d'accès aux fonctionnalités disponibles dans l'environnement web.

Adaptation au profil et au contexte :

Nous mettrons en place plusieurs profils utilisateurs fictifs afin de tester l'adaptation de l'interface selon différentes caractéristiques telles que l'âge, l'expérience, la langue ou les préférences.

Différentes stratégies de gestion du contexte seront également expérimentées afin de déterminer quelles informations doivent être transmises au LLM pour chaque interaction.

Tests et itérations :

Les fonctionnalités développées seront testées sur les scénarios définis précédemment. Les résultats permettront d'identifier les erreurs et les limites du système, puis d'améliorer progressivement le prototype.

Le développement suivra ainsi une démarche itérative de **développement, test, analyse et amélioration**. Les technologies et approches expérimentées seront documentées afin de faciliter leur comparaison lors de la phase de validation.

5.6. Bilan et perspectives

Enfin, nous réaliserons une synthèse des travaux réalisés et des résultats obtenus. Le bilan permettra de présenter les cas d'usage retenus, l'architecture mise en œuvre, les technologies expérimentées, les résultats obtenus, les difficultés rencontrées, les limites du prototype et les pistes d'amélioration.

L'objectif final sera ainsi de disposer d'un prototype fonctionnel permettant de démontrer le potentiel des Generative UIs et des interfaces AI Native dans une application bancaire.

6 Organisation

6.1 Présentation de l'équipe

Membre	Rôle organisationnel	Rôle technique
SAFTA Yassine	Chef de projet : vision, coordination, livrables	Tools & Backend Engineer : API, function calling, banque simulée
BEDOUIN Arnaud	Responsable planning & organisation : backlog, priorités, deadlines	Security / QA / Observability Engineer : tests, sécurité, monitoring
BADSSI Chaimae	Responsable communication client : interface avec Sopra Steria	AI / Agent Lead : prompts, orchestration agentique, contexte
BELKHIRIA Zeineb	Responsable documentation & livrables : CR, rapports, slides	GenUI / Frontend Engineer : moteur de rendu dynamique (React)
MOREAU Maxandre	Responsable qualité & risques : critères qualité, registre des risques	UX / Design System Engineer : parcours UX, composants autorisés
BARET Alexis	Responsable méthode agile : points d'équipe, suivi des tâches, rétrospectives	Architecture & Protocol Engineer : architecture globale, A2UI, AG-UI, WebMCP



6.3 Décomposition du projet en lots et tâches

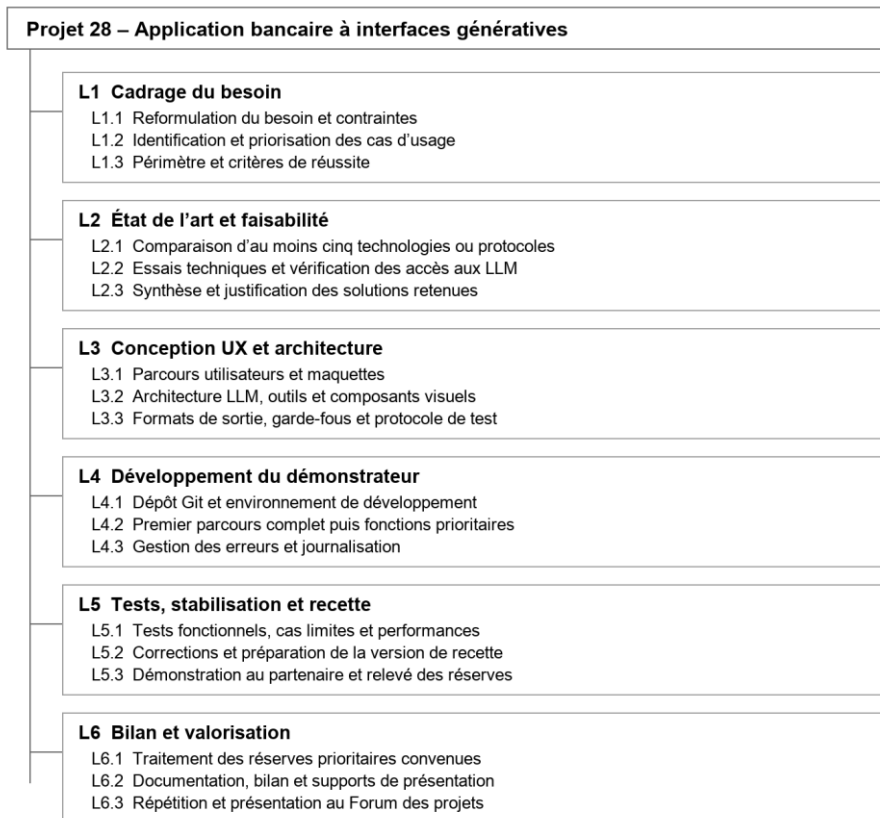


Figure 3 – Organigramme de la structure du projet (WBS).

6.4 Planification et critères de fin de lot

Lot	Période et durée	Livrable et critère de réussite
L1	21/09 – 02/10/2026 2 semaines	Document de cadrage et cas d'usage priorités ; périmètre soumis au partenaire pour validation (J1).
L2	28/09 – 16/10/2026 3 semaines	Comparaison d'au moins cinq technologies ou protocoles selon maturité, sécurité, intégration et performance ; essais et choix justifiés (consolidation à J2).
L3	12/10 – 23/10/2026 2 semaines	Maquettes, architecture, formats de sortie et protocole de test documentés et revus (J2).
L4	26/10 – 27/11/2026 5 semaines	Code versionné ; premier parcours démontré à J3, puis fonctions prioritaires intégrées et testables avant stabilisation.
L5	16/11 – 17/12/2026 au plus tard Environ 4 à 5 semaines	Anomalies bloquantes corrigées, résultats de tests disponibles et recette réalisée ; réserves consignées (J4 et J5).
L6	Après la recette – 22/01/2027 Environ 5 semaines	Bilan et documentation finalisés ; supports prêts à J6 ; présentation au forum à J7.

6.5 Ressources et suivi de l'avancement

Un responsable et un binôme seront désignés pour chaque lot selon les compétences et les disponibilités indiquées en section 6.1. Avant le lancement d'un lot, les tâches seront estimées en heures et réparties entre les membres. L'estimation sera ajustée lors des points d'avancement hebdomadaires.

Les accès aux LLM et les quotas fournis par Sopra Steria seront vérifiés pendant le lot L2. Le code sera conservé dans un dépôt Git et les comptes rendus, maquettes et documents dans un espace partagé.

La période du 30 novembre au 11 décembre est réservée en priorité aux corrections et à la préparation de la recette. Les développements secondaires seront reportés si les parcours essentiels ne sont pas stabilisés. La charge du lot L6 tiendra compte des congés de fin d'année.

6.6 Jalons et livrables

Le tableau 2 récapitule les rendez-vous du calendrier fourni et les jalons intermédiaires proposés. Les dates marquées d'un astérisque restent à confirmer. La date de recette doit être arrêtée avec le partenaire.

Tableau 2 – Jalons et livrables du projet.

ID	Intitulé	Description	Lot	Date	Moyen de livraison	Responsable
J0	Rencontre client	Clarification initiale du besoin.	L1	21/09/2026 Matin	Réunion et compte rendu partagé.	Chaimae
J1	Cadrage	Périmètre, cas d'usage prioritaires et critères de réussite.	L1	02/10/2026*	Document partagé ; validation sollicitée auprès du partenaire.	Maxandre
J2	Conception	Comparatif technique, maquettes et architecture retenue.	L2–L3	23/10/2026*	Documents partagés et revue technique.	Alexis
J3	Première version	Premier parcours bancaire complet.	L4	13/11/2026*	Démonstration ; code dans Git.	Alexis
J4	Version de recette	Fonctions prioritaires testées ; anomalies bloquantes corrigées.	L4–L5	11/12/2026*	Version identifiée dans Git et résultats de tests.	Yassine
J5	Recette du projet	Évaluation avec Sopra Steria et relevé des réserves.	L5	14/12/2026 Matin	Réunion technique, démonstration et compte rendu.	Arnaud

J6	Supports finalisés	Poster, vidéo, bilan et scénario de démonstration.	L6	15/01/2027*	Dépôt partagé et répétition interne.	Zeineb
J7	Forum des projets	Présentation des résultats et limites ; fin du projet.	L6	22/01/2027 Après-midi	Poster, vidéo et démonstration sur le campus.	Yassine

* Dates intermédiaires proposées.

6.7 Diagramme de Gantt et dépendances

Planning prévisionnel du projet

Sopra Steria • Interfaces bancaires génératives • 21 septembre 2026 – 22 janvier 2027

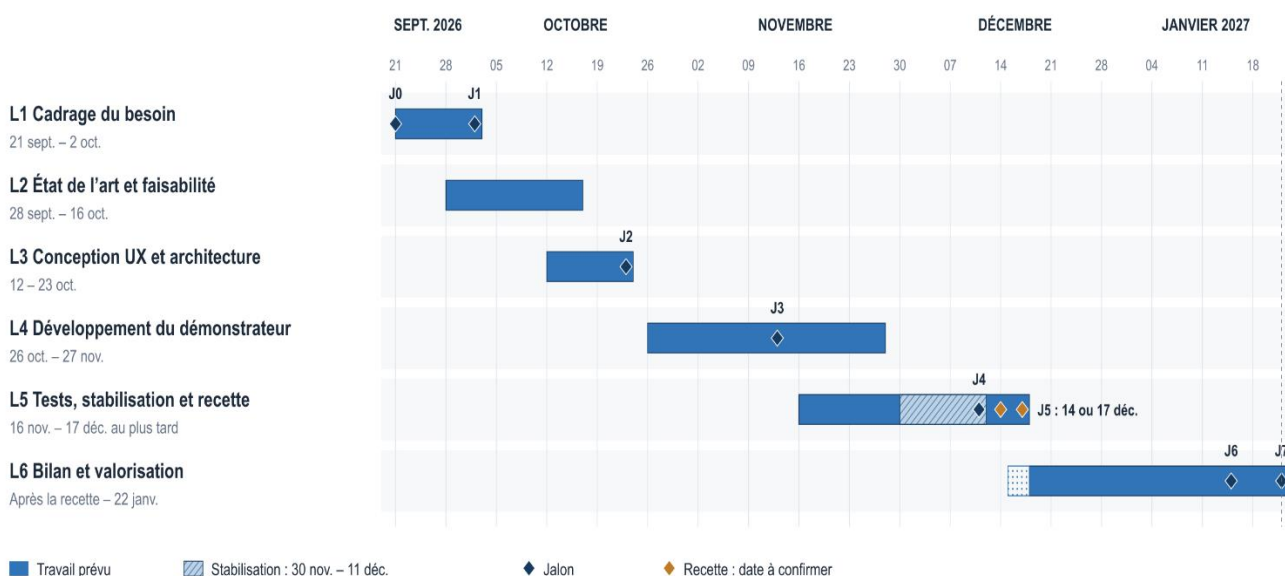


Figure 4 – Diagramme de Gantt prévisionnel, de septembre 2026 à janvier 2027.

6.8 Identification des risques

Une AMDEC simplifiée (analyse des modes de défaillance, de leurs effets et de leur criticité) est utilisée pour identifier les causes possibles de difficulté, leurs conséquences et les actions à prévoir. Elle couvre les dimensions techniques, organisationnelles, de communication, de ressources, de coût, de périmètre, de planification et de qualité. La confidentialité est traitée comme une contrainte du démonstrateur.

6.9 Méthode de cotation

Les cotations initiales sont des estimations à discuter avec l'équipe et l'encadrant. Trois notes de 1 à 3 sont attribuées à chaque risque. Une note de détection élevée signifie que le problème est difficile à détecter avant ses effets.

Note	Probabilité P	Gravité G	Difficulté de détection D
1	Faible	Effet limité, sans remise en cause du livrable essentiel.	Détection facile par les contrôles courants.
2	Moyenne	Dégradation d'un livrable ou retard rattrapable.	Détection nécessitant une revue ou un test ciblé.
3	Élevée	Objectif essentiel ou recette compromis.	Détection difficile avant manifestation des effets.

La criticité initiale est calculée par $C = P \times G \times D$. Pour ce projet, les scores de 1 à 4 sont faibles (vert), de 6 à 9 modérés (orange) et de 12 à 27 critiques (rouge). Ces seuils constituent une convention de priorisation interne au projet.

Faible	Modérée	Critique
C de 1 à 4	C de 6 à 9	C de 12 à 27

6.10 Modalités de suivi

Le tableau 3 présente l'AMDEC initiale. Les responsables indiqués sont des rôles à attribuer, et non des affectations nominatives déjà validées. Les actions proposées ne sont pas considérées comme réalisées. La criticité résiduelle sera réévaluée après leur mise en œuvre et leur vérification.

Revue	Échéance proposée	Mise à jour attendue
Initiale	Au cadrage, avant J1	Valider risques, notes, responsables et actions prioritaires.
Mi-projet	Autour du 09/11/2026	Recalculer P, G, D et C ; vérifier les actions et ajouter les risques nouveaux.
Avant recette	Au plus tard le 11/12/2026	Vérifier les risques bloquants et les solutions de repli.
Clôture	Avant le 22/01/2027	Clore ou transférer les risques ouverts ; consigner le retour d'expérience.

Le registre partagé comportera la date d'identification, le statut (ouvert, réduit, survenu ou clos), les notes actualisées et la preuve des actions réalisées. La date d'identification de chaque ligne sera renseignée lors de la validation initiale. Tout risque critique menaçant la recette sera signalé à l'encadrant et au partenaire concerné.

6.11 ADEC et tableau des risques

Tableau 3 – AMDEC initiale du projet.

P : probabilité ; G : gravité ; D : difficulté de détection ; C = P × G × D. Cotation proposée.
Vert : faible ; orange : modérée ; rouge : critique.

ID / Axe	Défaillance et effet	P/G/D	C	Détection prévue	Prévention	Réponse corrective	Responsable / Échéance
R1 Périmètre / planification	Trop de cas d'usage ou ajouts tardifs. Effet : Parcours essentiels inachevés à la recette.	3/3/2	18 Critique	Revue hebdomadaire du périmètre et des tâches restantes.	Définir un démonstrateur minimal ; valider les priorités avec Sopra Steria.	Reporter les fonctions secondaires en conservant un parcours complet.	Coordination Avant J1 puis chaque semaine
R2 Technique	Protocole ou bibliothèque difficile à intégrer ; documentation ou compatibilité insuffisante. Effet : Blocage de l'architecture et retard du prototype.	3/2/2	12 Critique	Essais ciblés en L2 et revue d'intégration.	Comparer les solutions ; limiter le nombre de protocoles intégrés.	Retenir une solution plus simple ; documenter les limites.	Architecture Avant J2
R3 Ressources / externe / coût	Accès LLM retardé, service indisponible ou quota épuisé. Effet : Arrêt des développements ou des tests avec le modèle.	2/3/1	6 Modéré	Test d'accès précoce et suivi de consommation.	Confirmer accès, quotas et modalités d'usage ; limiter les appels inutiles.	Utiliser des réponses simulées clairement signalées ; solliciter le partenaire.	Backend IA L2 puis suivi continu
R4 Technique / qualité	Sortie du LLM erronée ou non conforme au format attendu. Effet : Interface invalide, données incohérentes ou action inappropriée.	3/3/2	18 Critique	Validation des schémas et tests de limites.	Limiter les composants ; autoriser et contrôler formats, données et actions.	Bloquer la sortie invalide ; afficher un état de repli et journaliser l'erreur.	IA frontend et L3 à L5
R5 Confidentialité / sécurité	Données sensibles ou clés exposées dans les appels, journaux ou le dépôt Git. Effet : Divulgaration d'informations ; interruption du démonstrateur.	2/3/3	18 Critique	Revue du code et des journaux des secrets.	Données fictives ; clés hors du code ; contrôle des accès et des traces.	Suspendre le flux ; révoquer les secrets compromis ; prévenir le partenaire.	Architecture Avant intégration puis revues

6.12 AMDEC et tableau des risques — suite

Tableau 3 – AMDEC initiale du projet (suite).

P : probabilité ; G : gravité ; D : difficulté de détection ; C = P × G × D. Cotations proposées.
Vert : faible ; orange : modérée ; rouge : critique.

ID / Axe	Défaillance et effet	P/G/D	C	Détection prévue	Prévention	Réponse corrective	Responsable / Echéance
R6 UX / performance	Génération trop lente ou composants visuels instables. Effet : Parcours difficile à utiliser ou démonstration peu convaincante.	2/2/2	8 Modéré	Mesure des temps de réponse et des essais de parcours.	Règles visuelles communes ; composants stables ; budget de latence à convenir.	Simplifier les écrans ; réduire les appels ; conserver un affichage de repli.	UX et frontend L3 à L5
R7 Communication / besoin	Attentes mal comprises ou décisions du partenaire non consignées. Effet : Prototype en décalage avec les critères de recette.	2/3/2	12 Critique	Revue régulière de maquettes et démonstrations.	Valider le périmètre et les critères ; partager les comptes rendus et décisions.	Reprioriser les travaux avec le partenaire ; ajuster le périmètre convenu.	Relation client J1, J2 et J3
R8 Organisation / ressources	Indisponibilité d'un membre ou compétence critique détenue par une seule personne. Effet : Tâche bloquée et retard de livraison.	2/2/2	8 Modéré	Point de disponibilité et revue des tâches bloquées.	Binôme sur les composants critiques ; documentation et revues croisées.	Réaffecter les tâches ; alerter l'encadrant si une échéance est menacée.	Coordination Chaque semaine
R9 Externe / démonstration	Panne réseau, service externe indisponible ou régression le jour de la présentation. Effet : Démonstration en direct impossible.	2/3/2	12 Critique	Répétition sur l'environnement prévu et tests avant présentation.	Conserver une version stable ; préparer la vidéo et le scénario de secours.	Présenter l'enregistrement en indiquant qu'il s'agit d'une vidéo.	Tests et démonstration Avant J5 et J7
R10 Qualité documentaire	Documentation ou support de présentation non mis à jour après une modification. Effet : Reprise du prototype et compréhension du bilan plus difficiles.	2/1/2	4 Faible	Relecture croisée avec la version finale du code.	Mettre à jour la documentation à chaque jalon ; désigner un relecteur.	Corriger les supports et consigner les limites restantes.	Documentation A chaque jalon et avant J6

7 Validation du projet

La validation du projet repose sur les critères de réussite définis en section 2.4. Chaque objectif est associé à une preuve concrète et à un jalon du planning (Tableau 5), comme le montre le Tableau 7.

Tableau 7. Modalités de validation des objectifs.

Objectif	Modalité de validation	Preuve / livrable	Validé par	Jalon
1 – Compréhension du problème	Reformulation du besoin et questions de clarification présentées au partenaire	Slides + CR de la réunion du 21/09	Client	J0 (fait)
2 – État de l'art	Au moins 5 technologies ou protocoles évalués sur 4 critères pondérés (maturité, sécurité, intégration, performance)	Section état de l'art du document de cadrage	Client + tuteur	J2
3 – Cas d'usage	Au moins 5 cas d'usage analysés (utilité, faisabilité, risques), dont 3 à 4 retenus pour le prototype	Document de cadrage + CR de validation	Client	J1, J2
4 – Expérimentations	Cas d'usage retenus fonctionnels de bout en bout ; tests passants ; architecture documentée	Code versionné sur Git (tags V1, V2), README, document d'architecture, rapport de tests	Client (recette)	J3 à J6
5 – Bilan	Démonstration en direct et rapport présentant apports, limites et pistes de poursuite	Rapport de bilan, démo, poster, vidéo	Client + tuteur + jury	J7, J8

En complément, la validation du projet s'appuie sur les éléments suivants, qui correspondent aux livrables attendus par Sopra Steria :

- **Documents de cadrage (20 %)** : le présent document, les CR de réunions et le cahier des charges fonctionnel.
- **Documents d'architecture (10 %)** : les diagrammes, les choix technologiques justifiés et les interfaces entre composants.
- **Code des prototypes versionné (50 %)** : le dépôt Git avec l'historique, les tags de version, un README d'installation, des tests automatisés et des traces d'observabilité (latence, tokens, erreurs).
- **Présentations et démonstrations (20 %)** : les démos intermédiaires (V1, V2), la recette de décembre et le forum de janvier.

La recette (J6) suivra un scénario de démonstration défini à l'avance. Chaque cas d'usage retenu y sera joué de bout en bout, avec au moins un cas limite, par exemple une demande de virement ambiguë ou une tentative de prompt injection. Cela permettra de vérifier les fonctions et critères chiffrés de la section 4.

8 Références bibliographiques

- https://www.linkedin.com/posts/enzo-melliti_ai-first-in-europe-white-paper-activity-7463224296425910272-pdLf?utm_source=share&utm_medium=member_desktop&rcm=ACoAAAgc4lgBoRW7QJ1fOZH7-GcdU1lbBEcQeTo
- https://www.linkedin.com/posts/marcelvanoost_australia-just-saw-its-first-ai-initiated-activity-7470248368687190017-8BNR?utm_source=share&utm_medium=member_desktop&rcm=ACoAAAgc4lgBoRW7QJ1fOZH7-GcdU1lbBEcQeTo
- AIR | AI by Revolut | Revolut FR